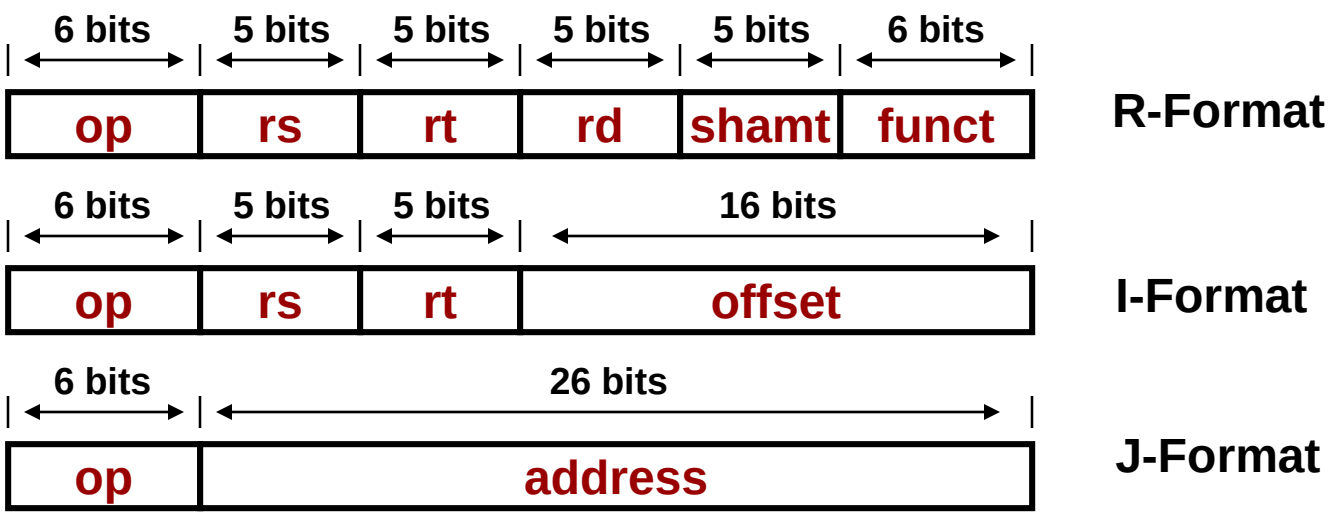
The Processor: Datapath and Control

Implementing MIPS

Implementation of the MIPS instruction set
Simplified to contain *only*

- arithmetic-logic instructions: `add, sub, and, or, slt`
- memory-reference instructions: `lw, sw`
- control-flow instructions: `beq, j`

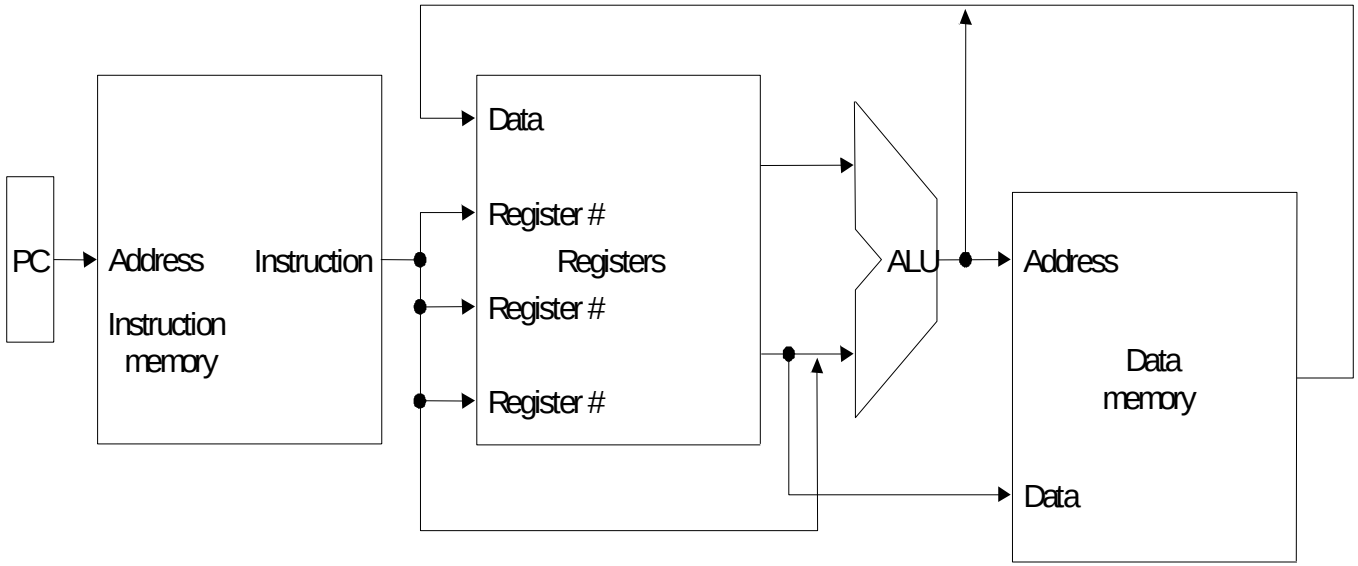


Implementing MIPS: the Fetch/Execute Cycle

High-level abstract view of *fetch/execute* implementation

- use the program counter (PC) to read instruction address
- *fetch* the instruction from memory and increment PC
- use fields of the instruction to select registers to read *execute* depending on the instruction

repeat…



Overview: Processor Implementation

Single Cycle

- perform each instruction in 1 clock cycle
- clock cycle must be long enough for slowest instruction; therefore,
- disadvantage: only as fast as slowest instruction

Multi-Cycle

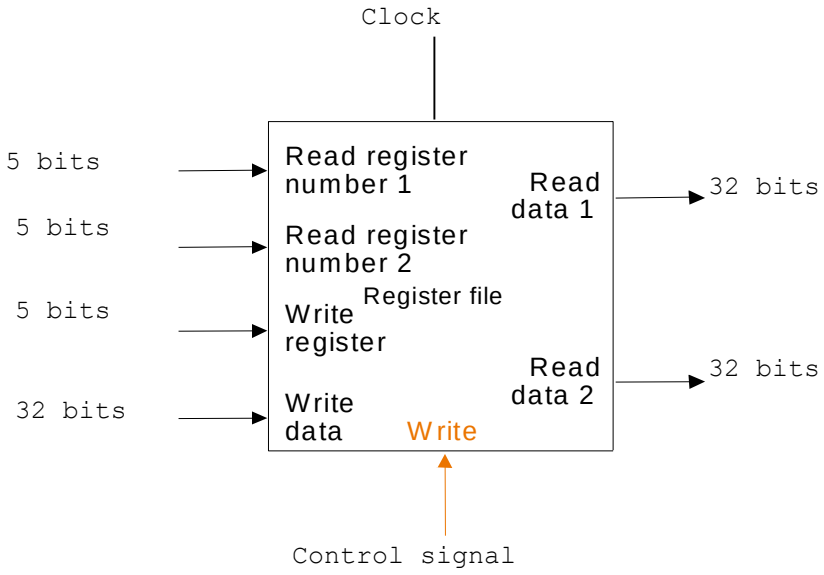
- break fetch/execute cycle into multiple steps
- perform 1 step in each clock cycle
- advantage: each instruction uses only as many cycles as it needs

Pipelined

- execute each instruction in multiple steps
- perform 1 step / instruction in each clock cycle
- process multiple instructions in parallel – assembly line

State Elements on the Datapath: Register File

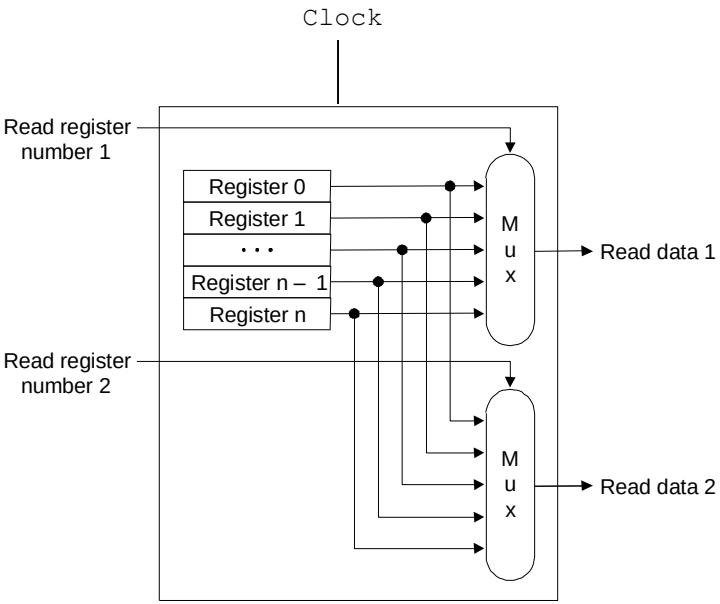
Registers are implemented with arrays of D-flipflops



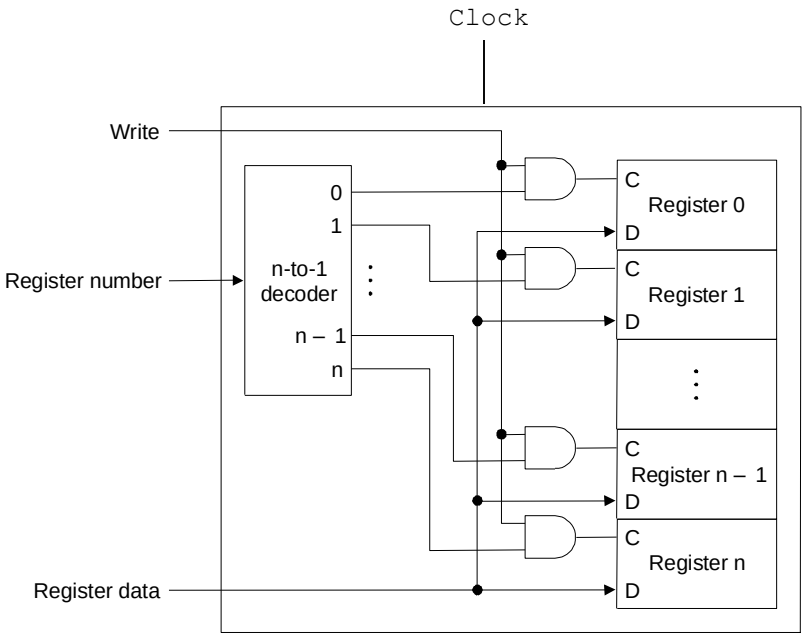
Register file with two read ports and one write port

Ex: Register File

Port implementation:

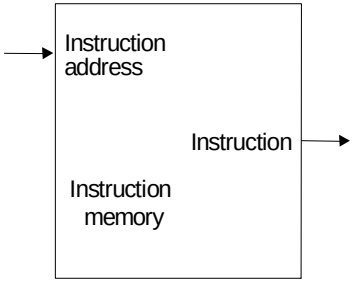


Read ports are implemented with a pair of multiplexors for 32 registers

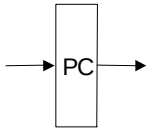


Write port is implemented using a decoder – 5-to-32 decoder for 32 registers. Clock is relevant to write as register state may change only at clock edge

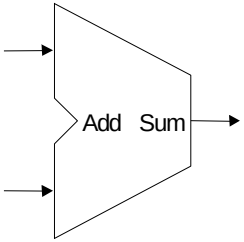
Datapath: Instruction Fetch & PC Increment



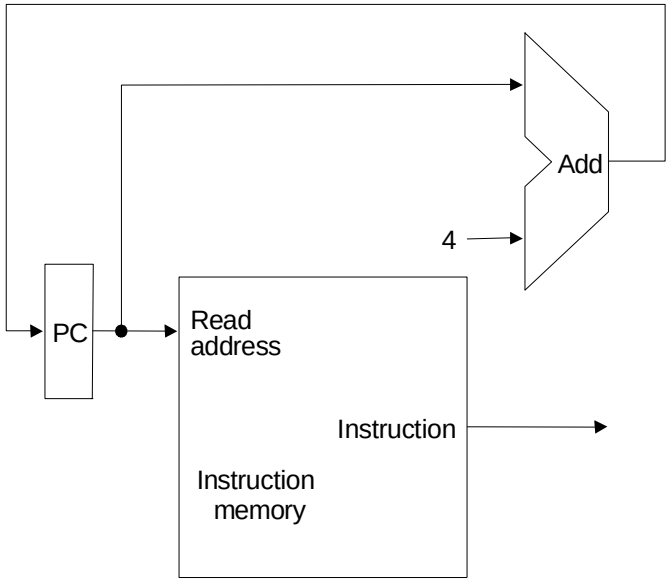
a. Instruction memory



b. Program counter



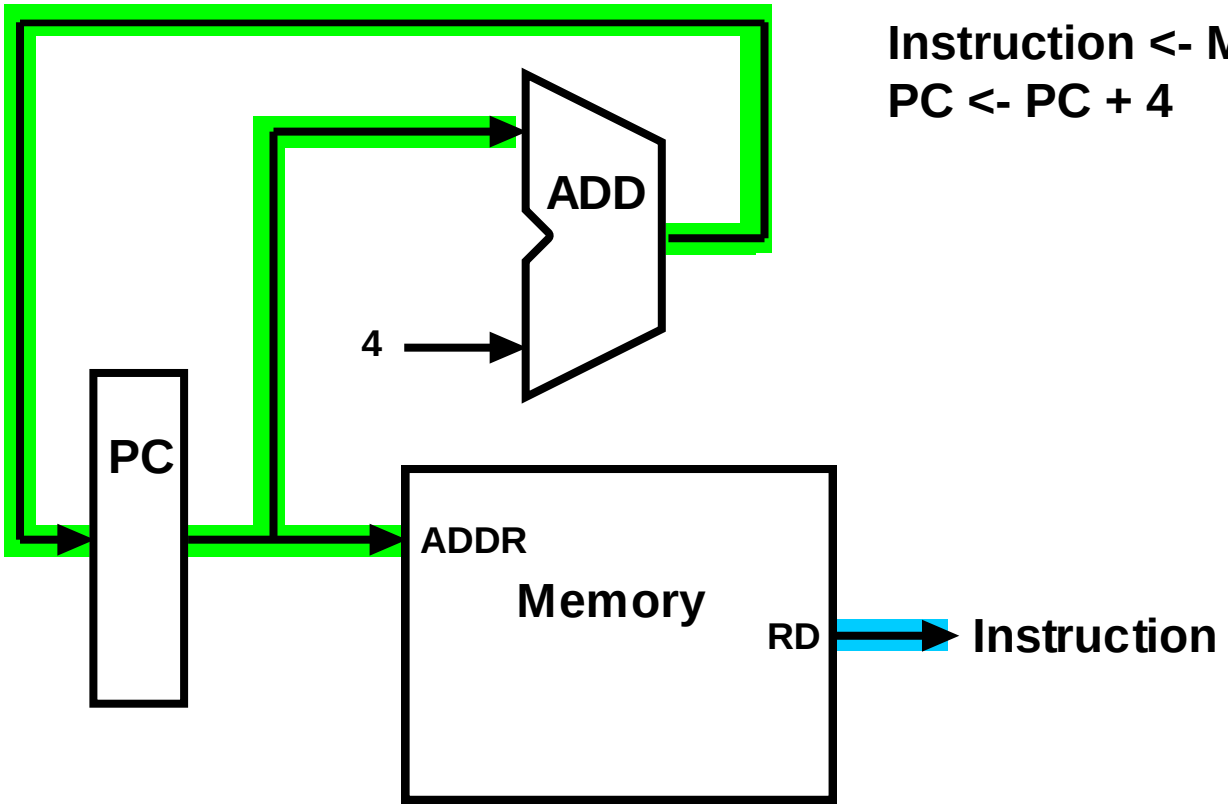
c. Adder



Three elements used to store and fetch instructions and increment the PC

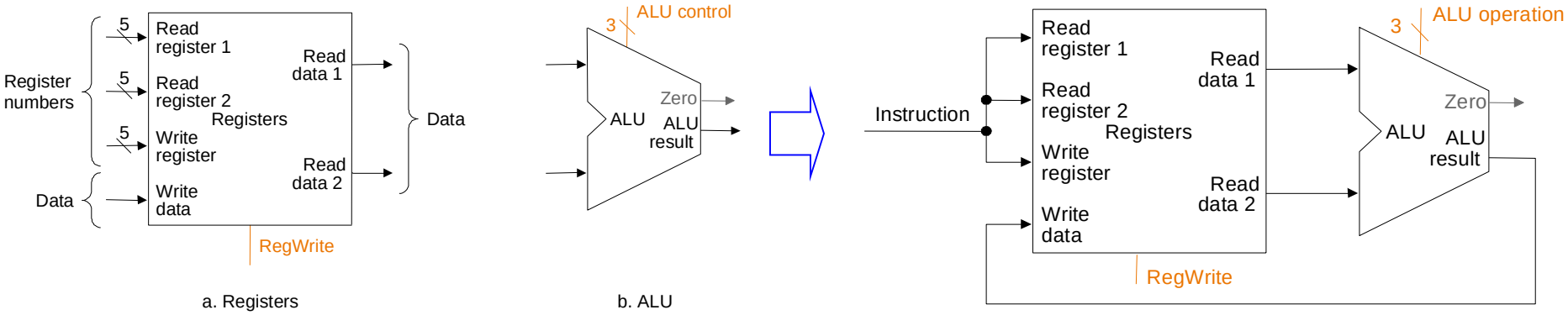
Datapath

Animating the Datapath



Instruction <- MEM[PC]
PC <- PC + 4

Datapath: R-Type Instruction



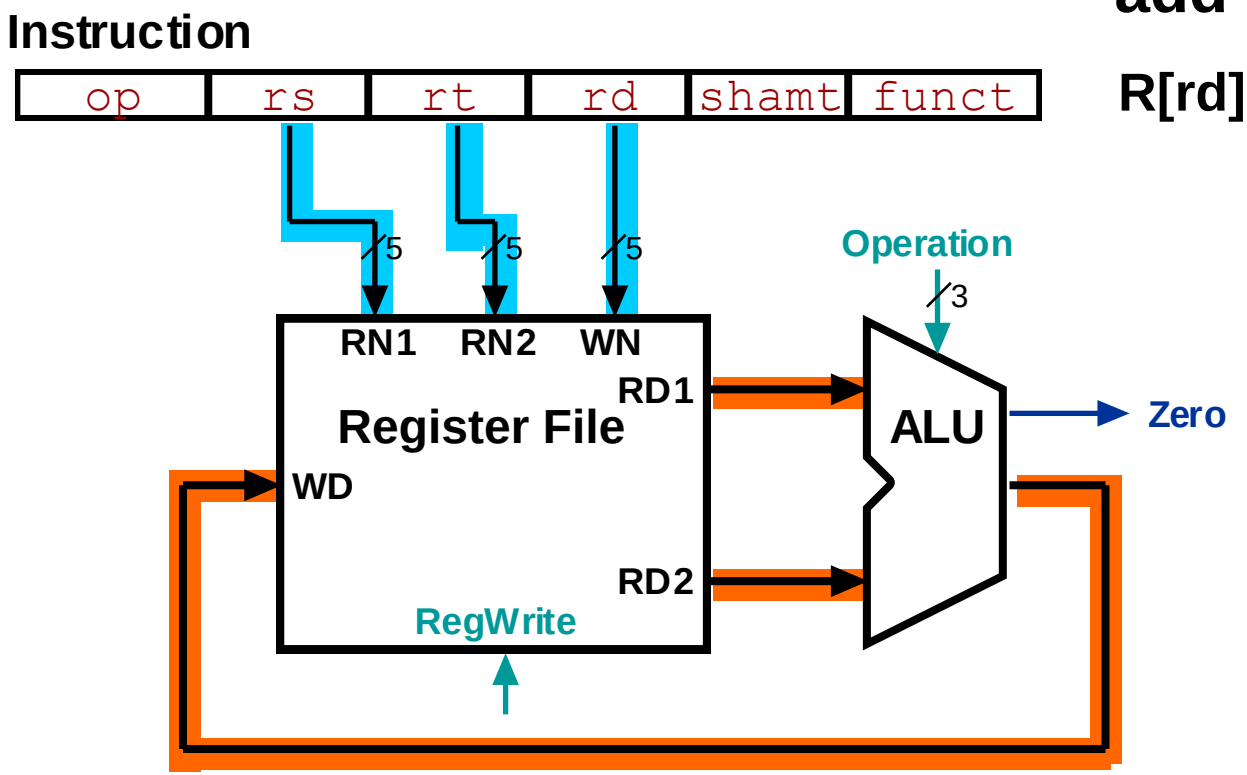
Two elements used to implement R-type instructions

Datapath

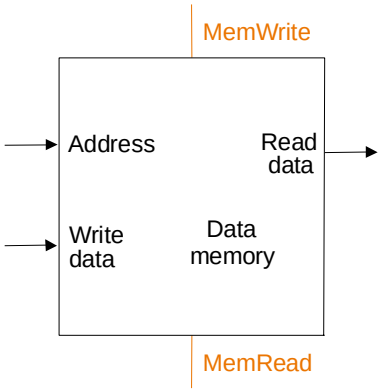
Animating the Datapath

add rd, rs, rt

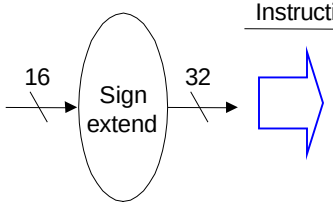
R[rd] ← R[rs] + R[rt];



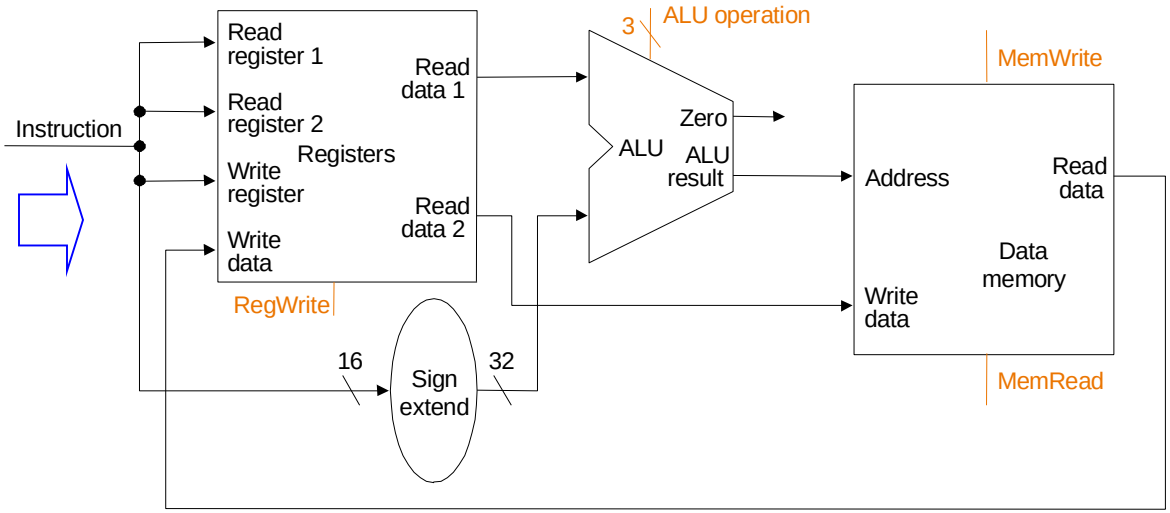
Datapath: Load/Store Instruction



a. Data memory unit



b. Sign-extension unit



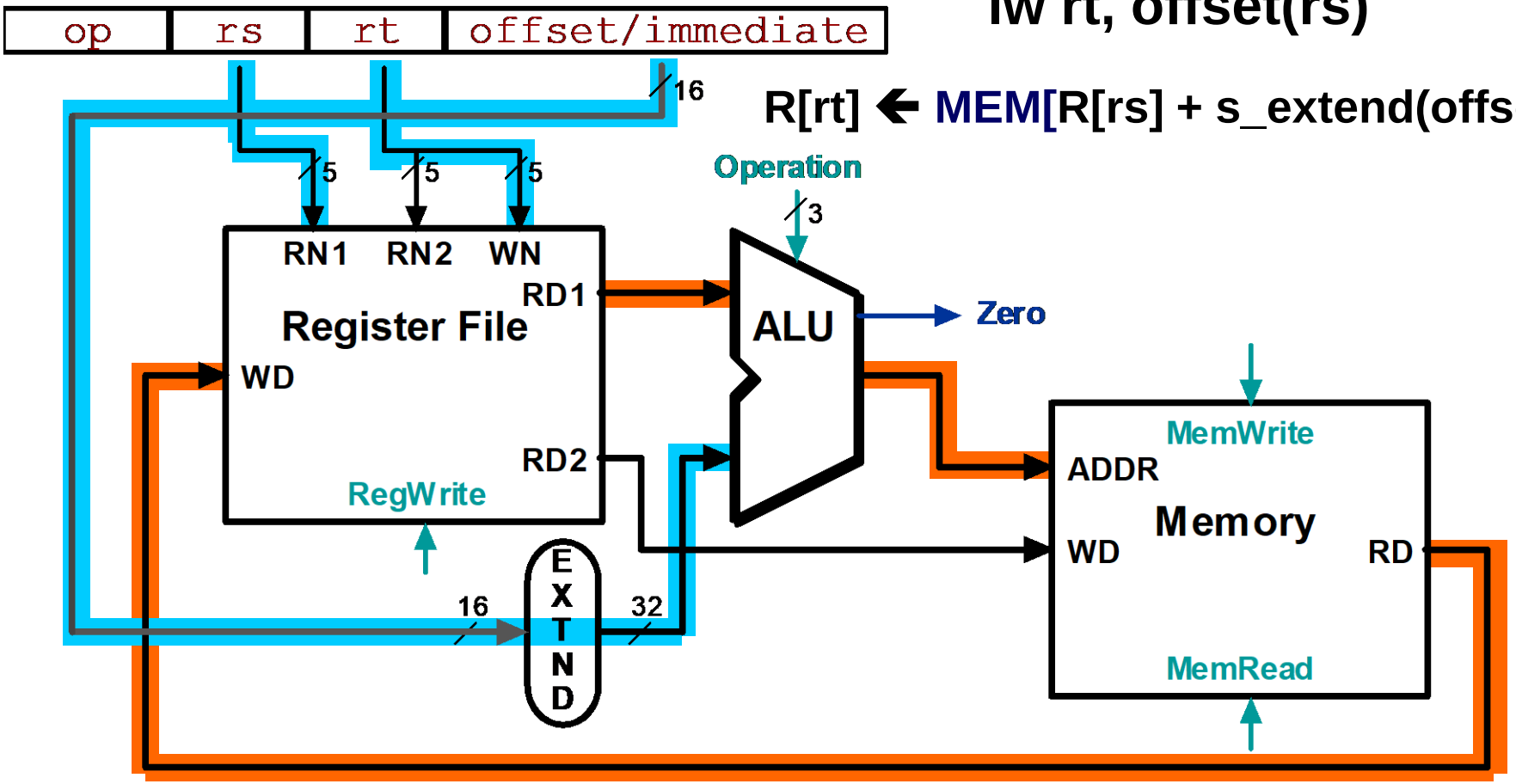
Two additional elements used
To implement load/stores

Datapath

Animating the Datapath

lw rt, offset(rs)

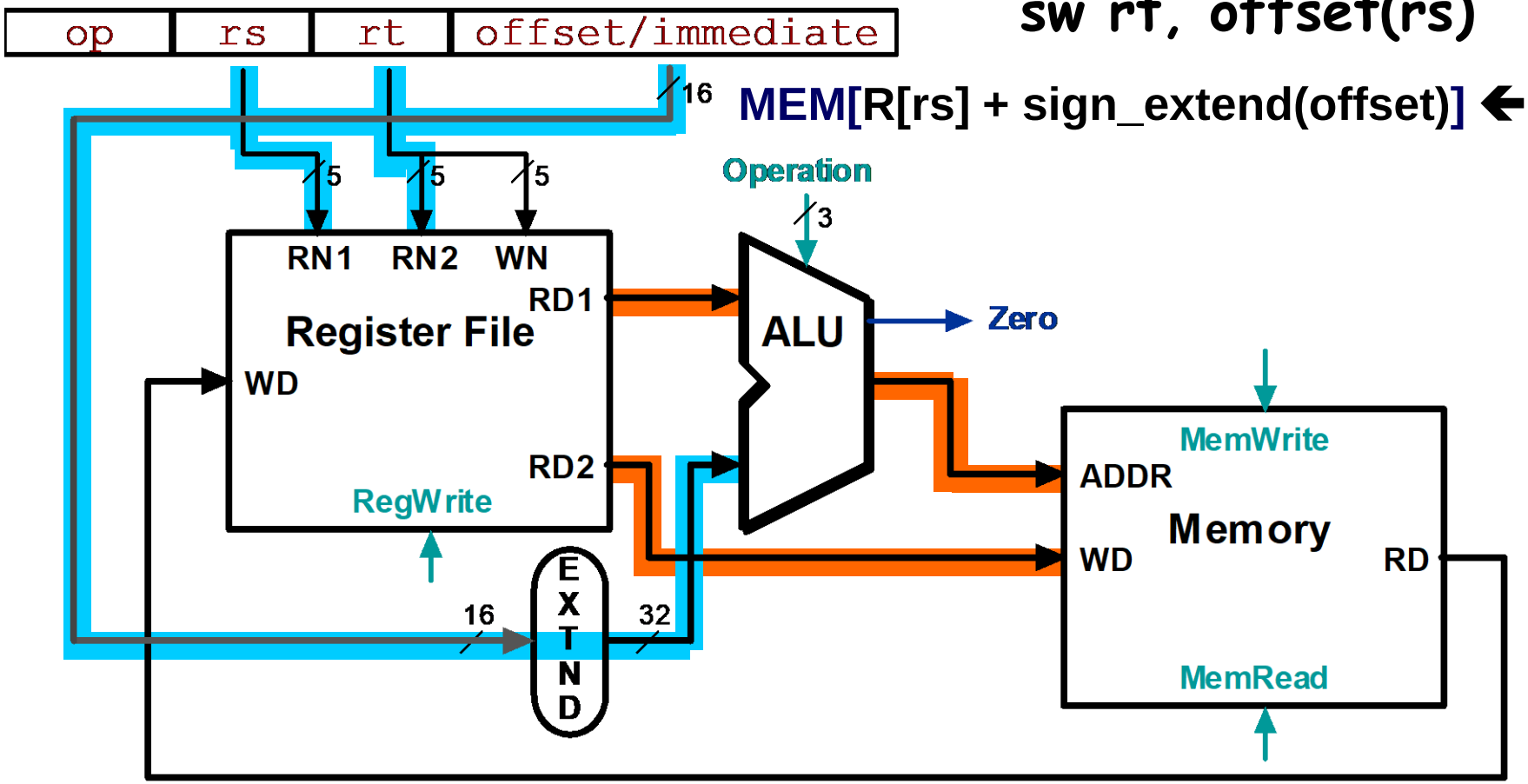
R[rt] ← MEM[R[rs] + s_extend(offset)];



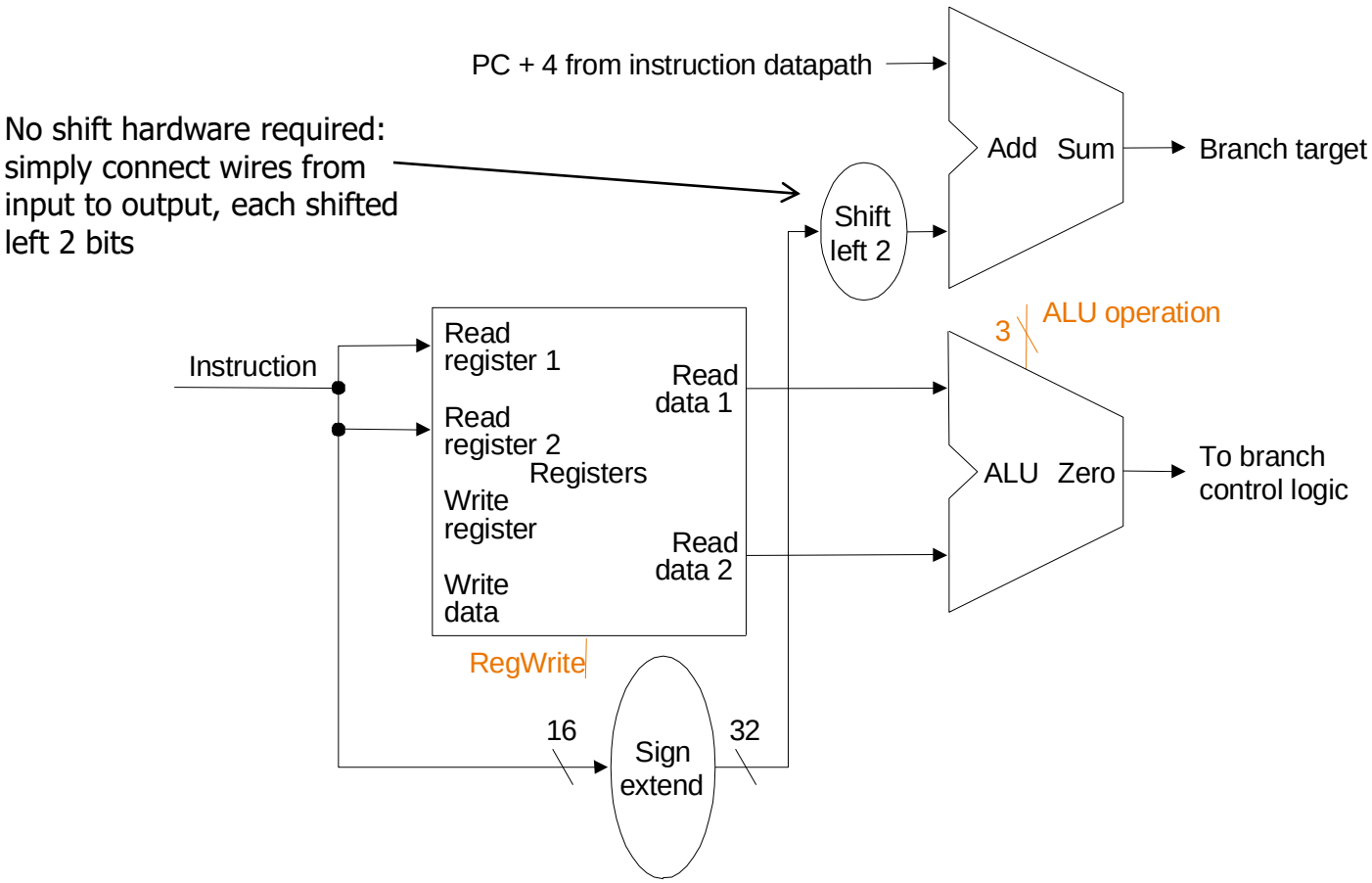
Animating the Datapath

sw rt, offset(rs)

MEM[R[rs] + sign_extend(offset)] ← R[rt]

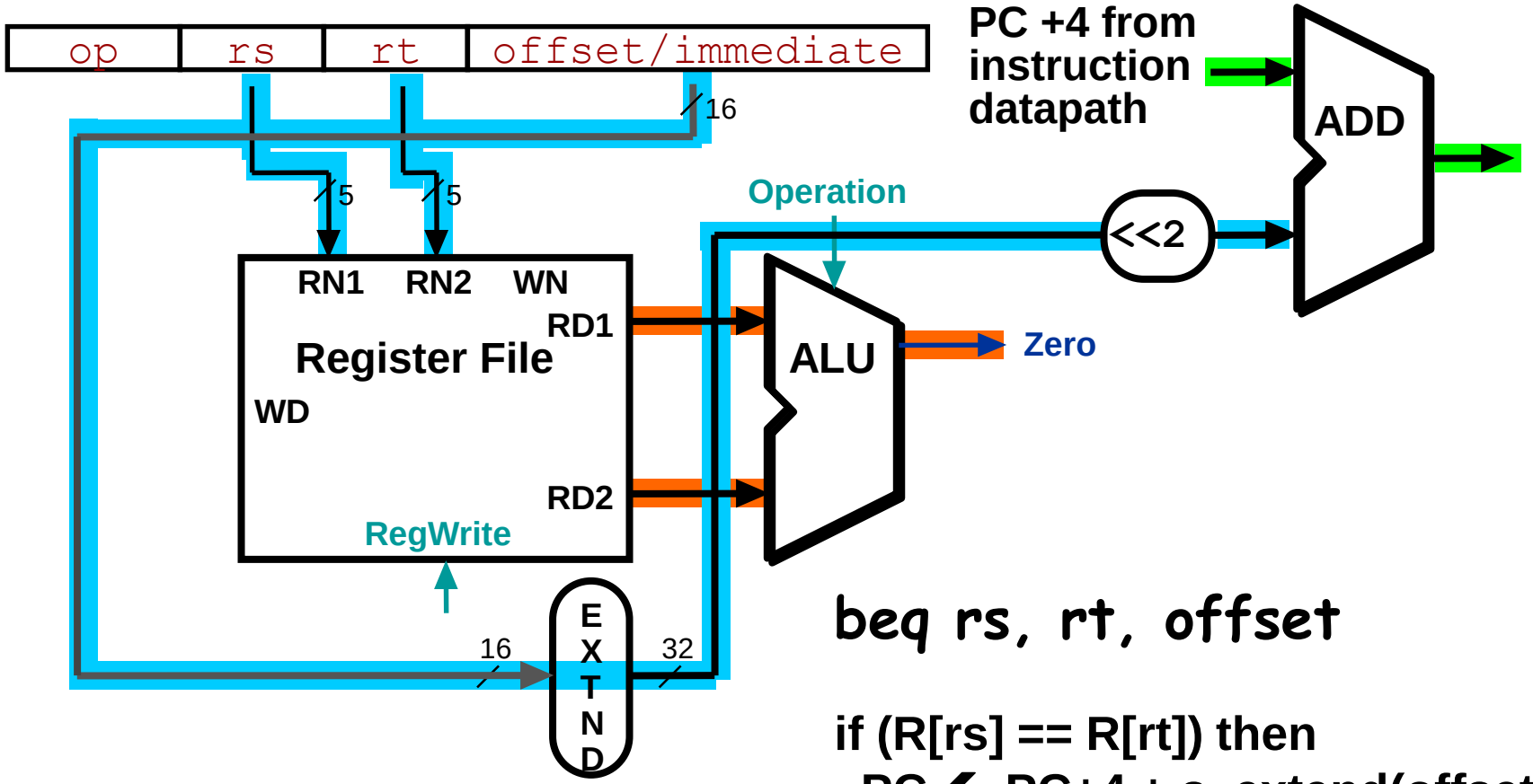


Datapath: Branch Instruction



Datapath

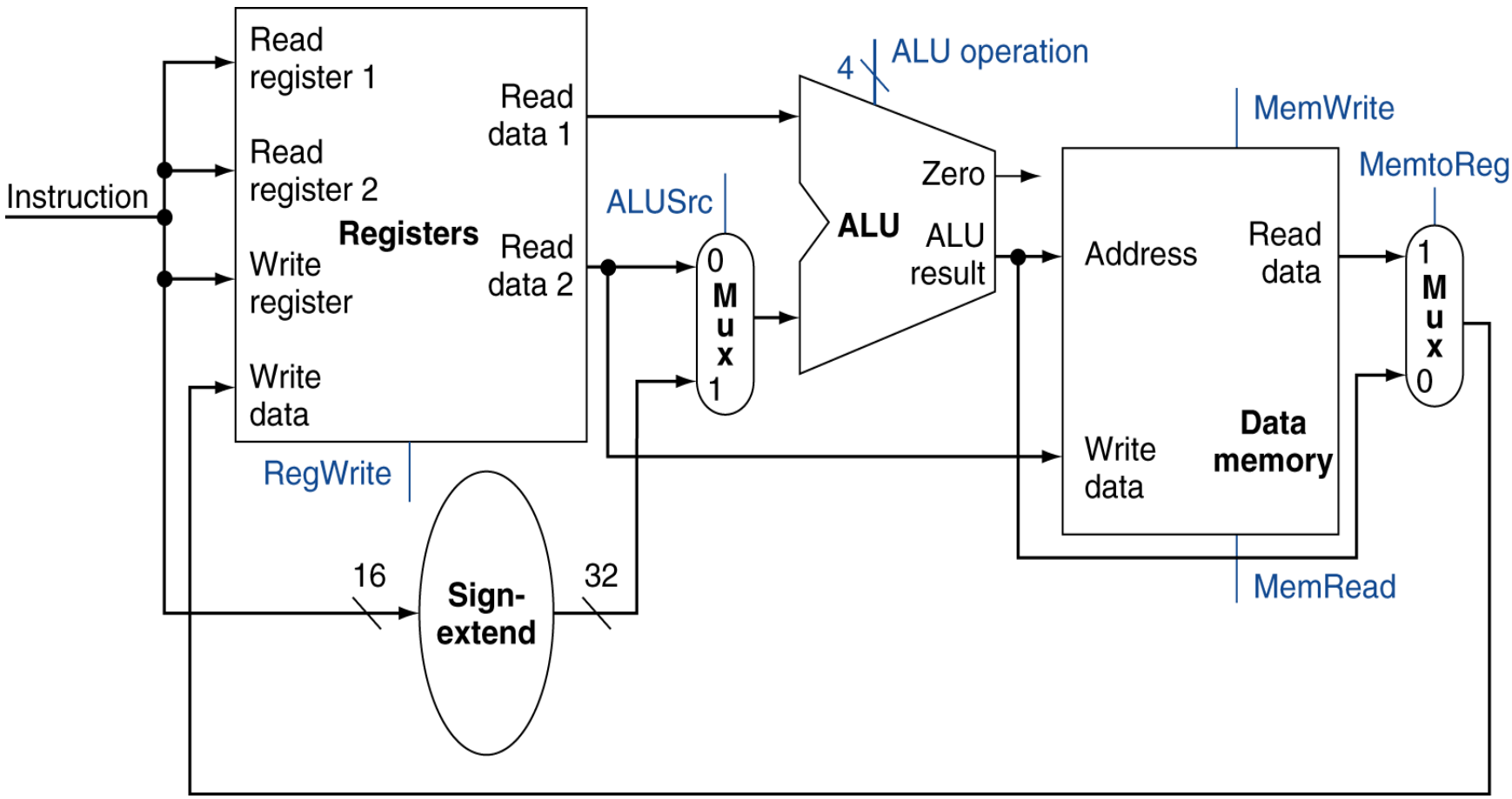
Animating the Datapath



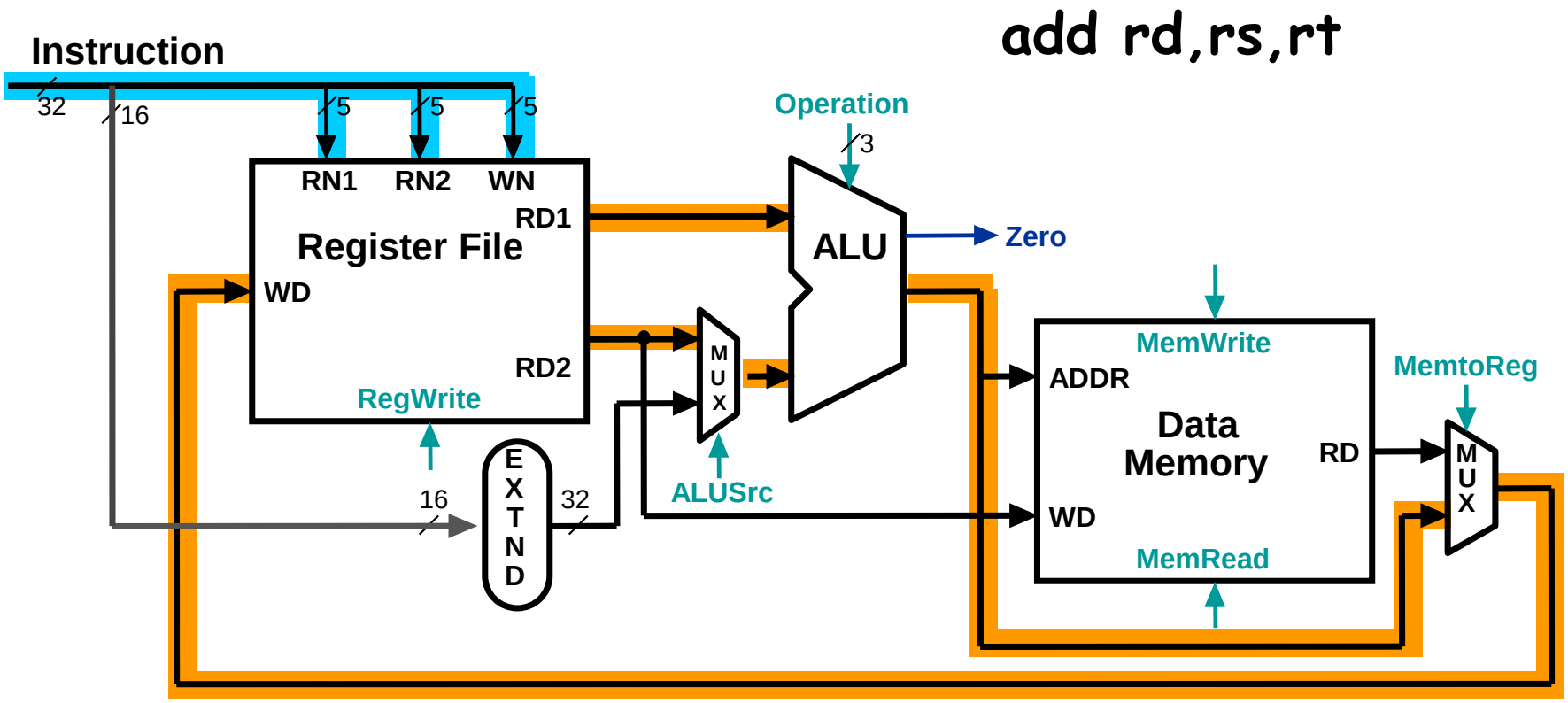
beq rs, rt, offset

if (R[rs] == R[rt]) then
PC ← PC+4 + s_extend(offset<<2)

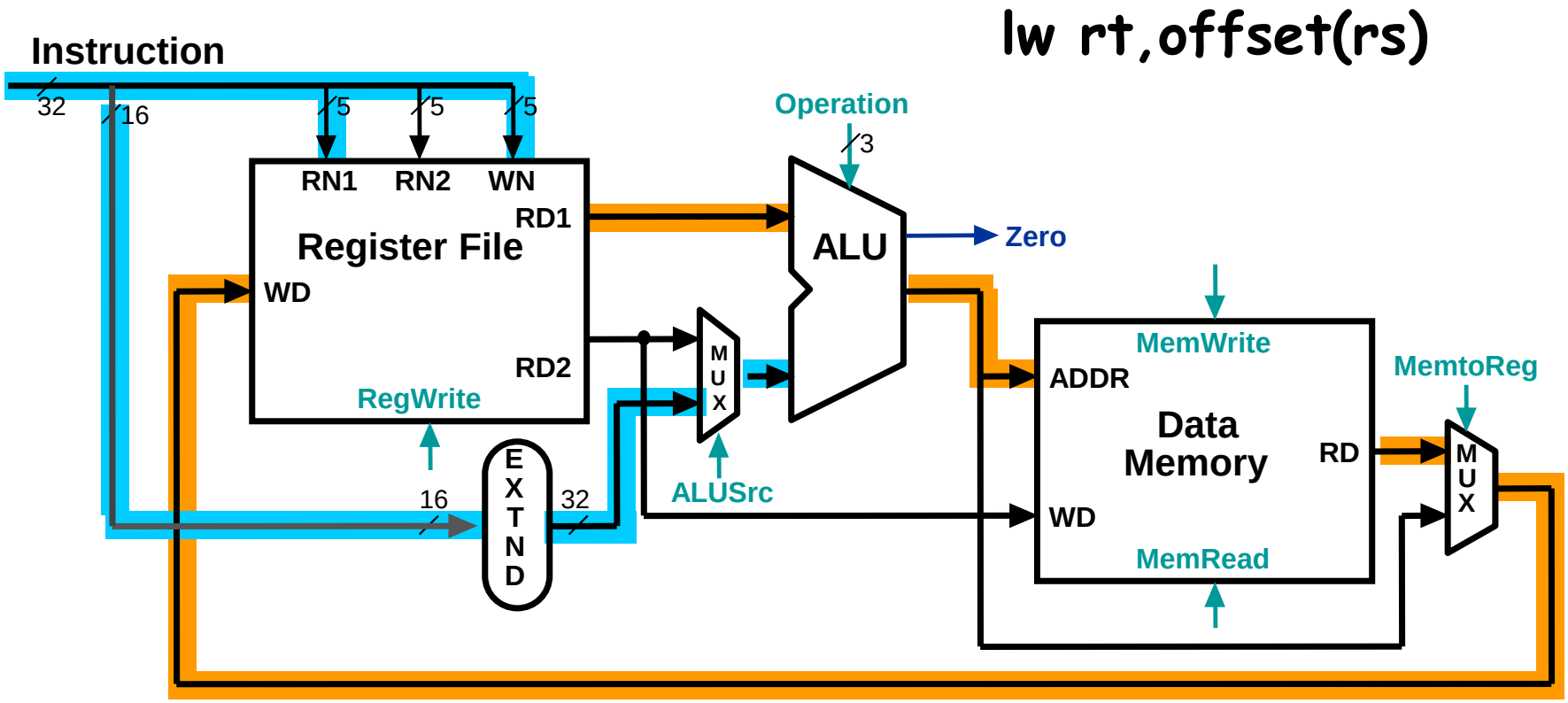
MIPS Datapath I: Single-Cycle



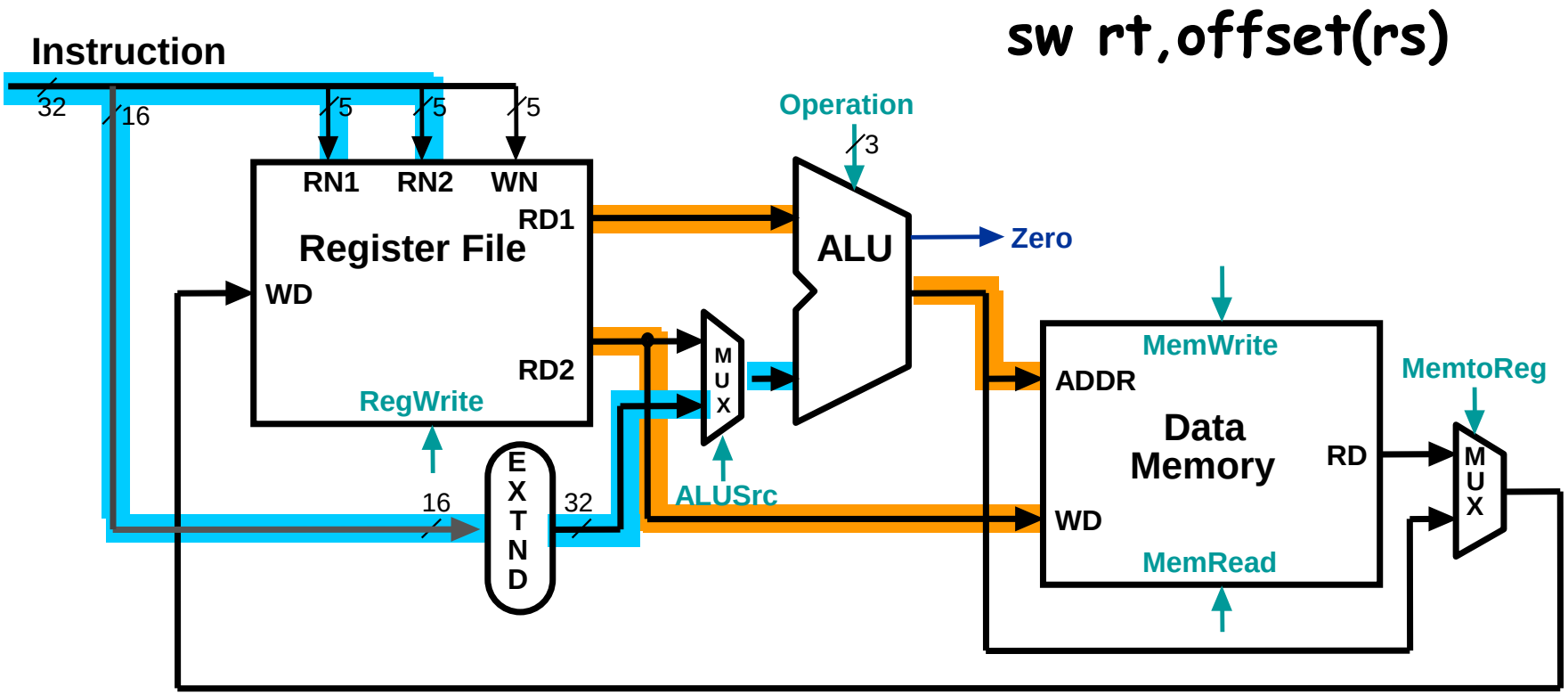
Animating the Datapath: R-type Instruction



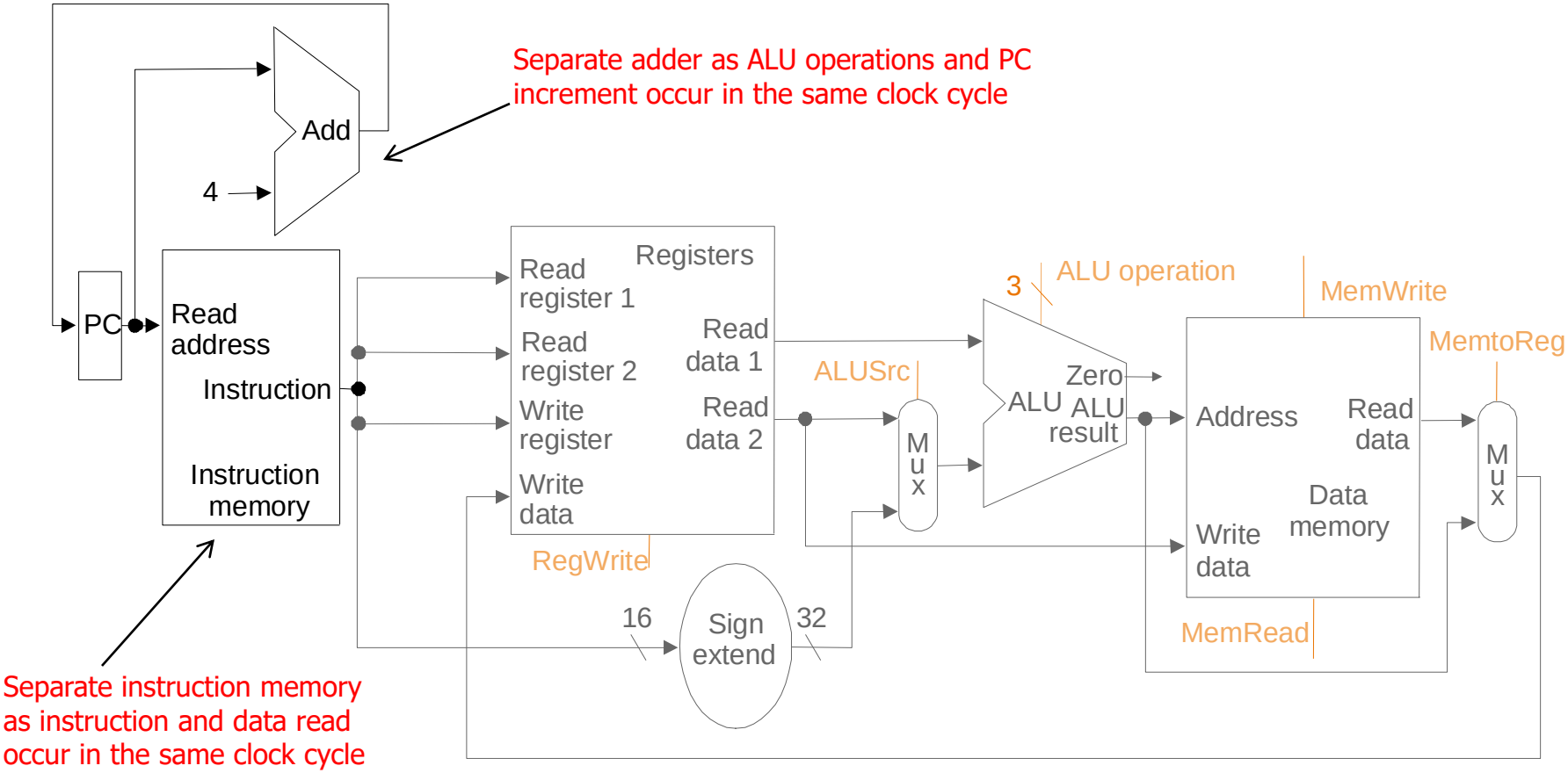
Animating the Datapath: Load Instruction



Animating the Datapath: Store Instruction

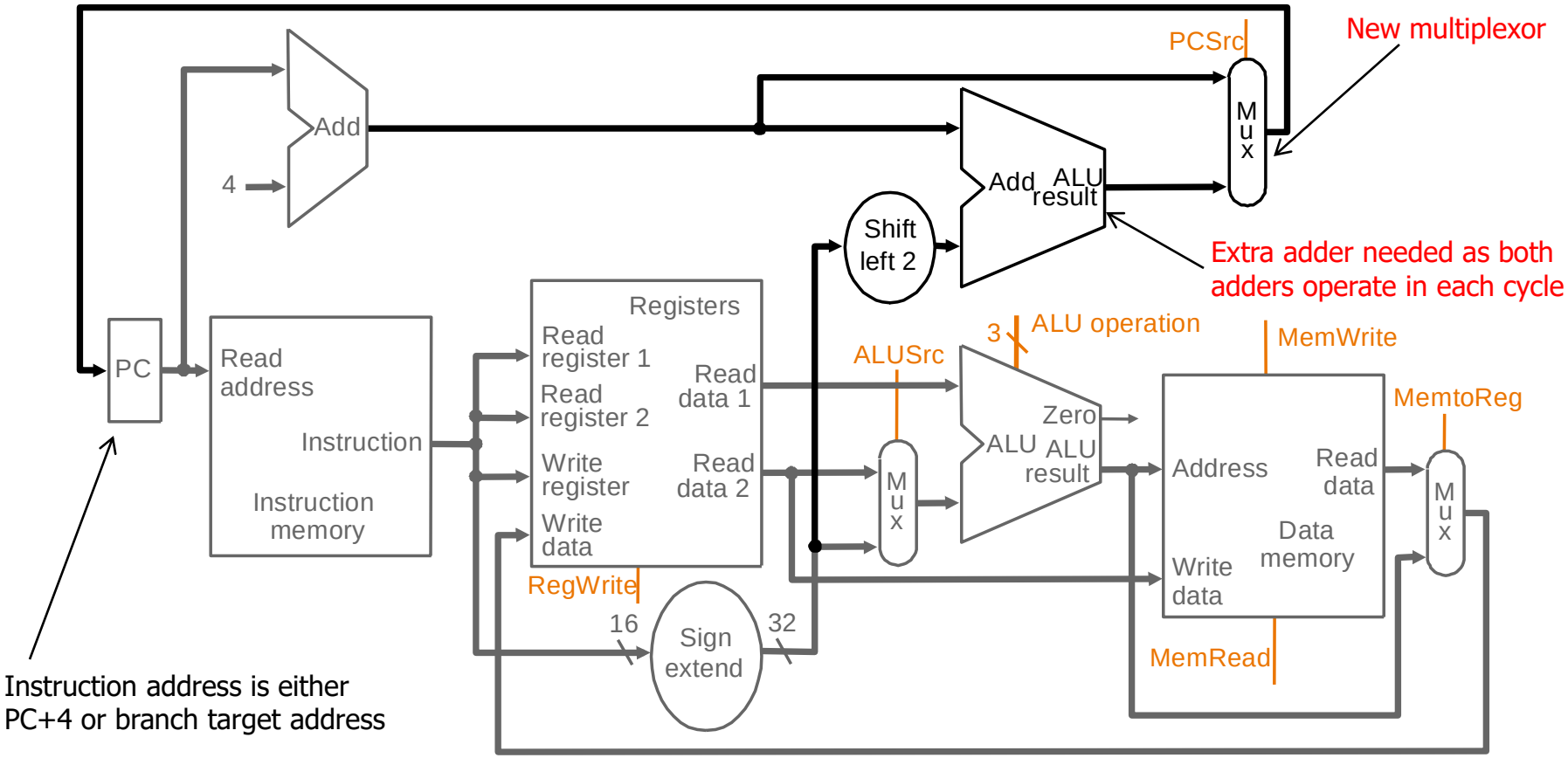


MIPS Datapath II: Single-Cycle



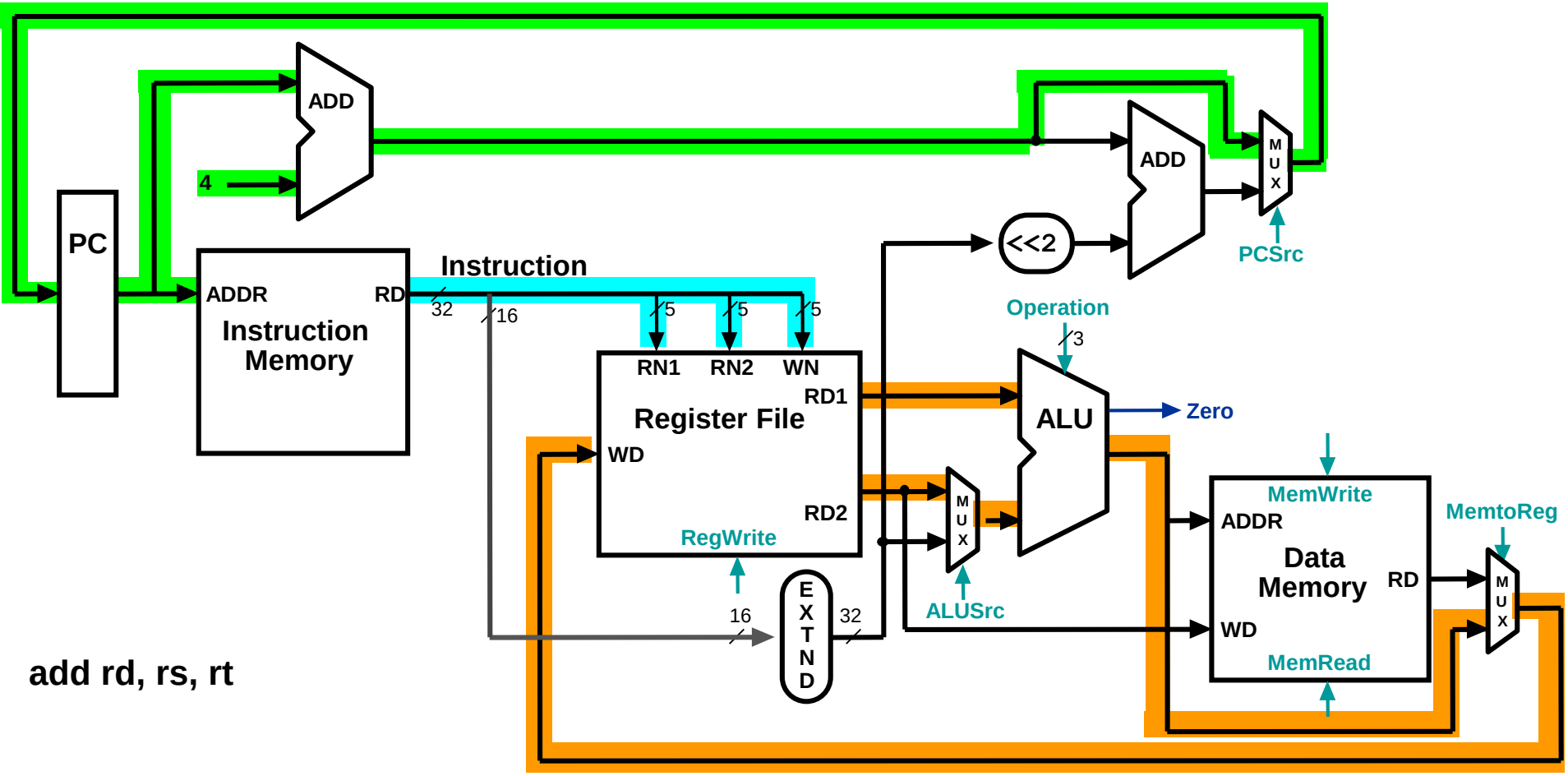
Adding instruction fetch

MIPS Datapath III: Single-Cycle



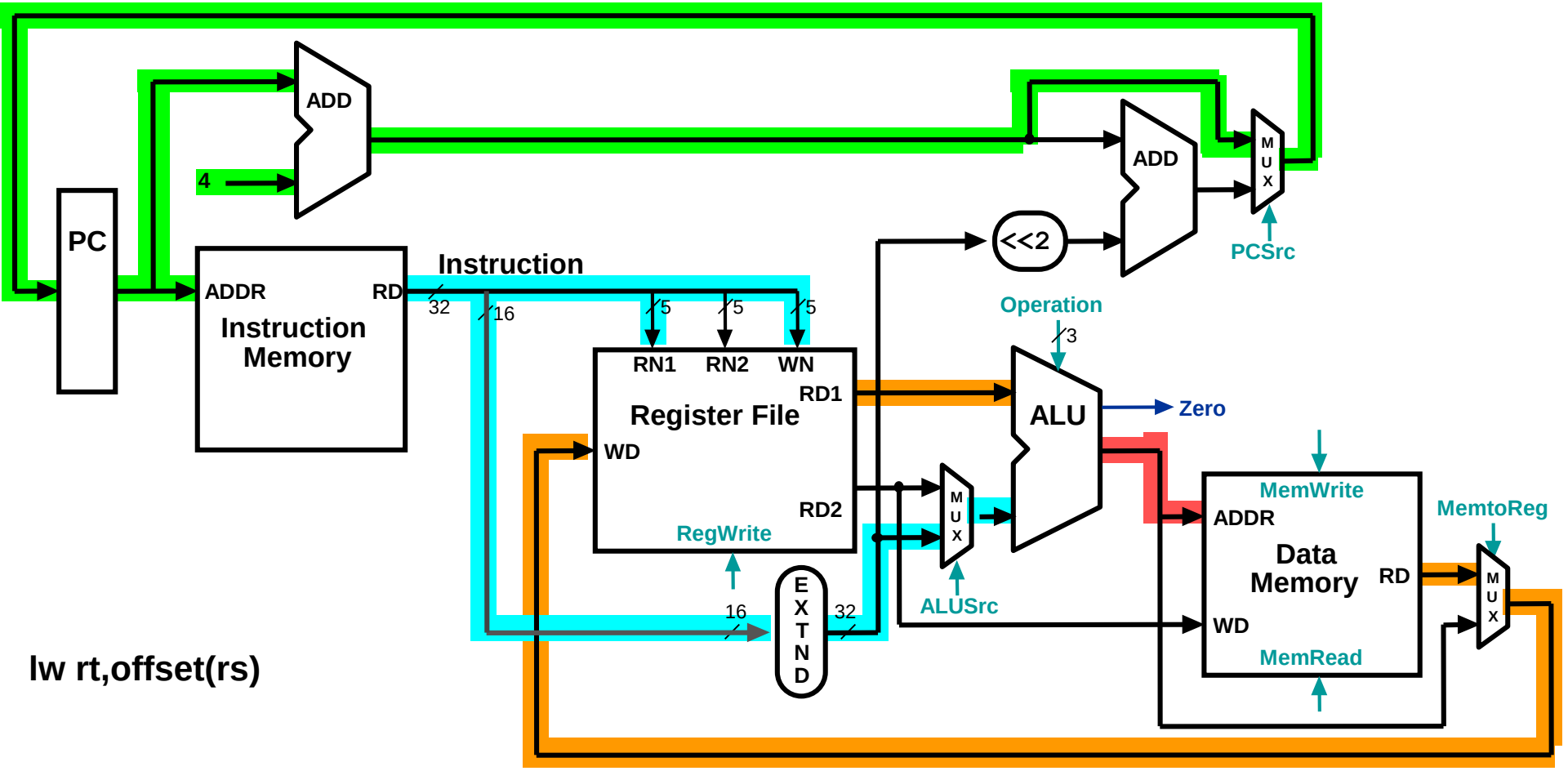
Adding branch capability and another multiplexor

Datapath Executing `add`



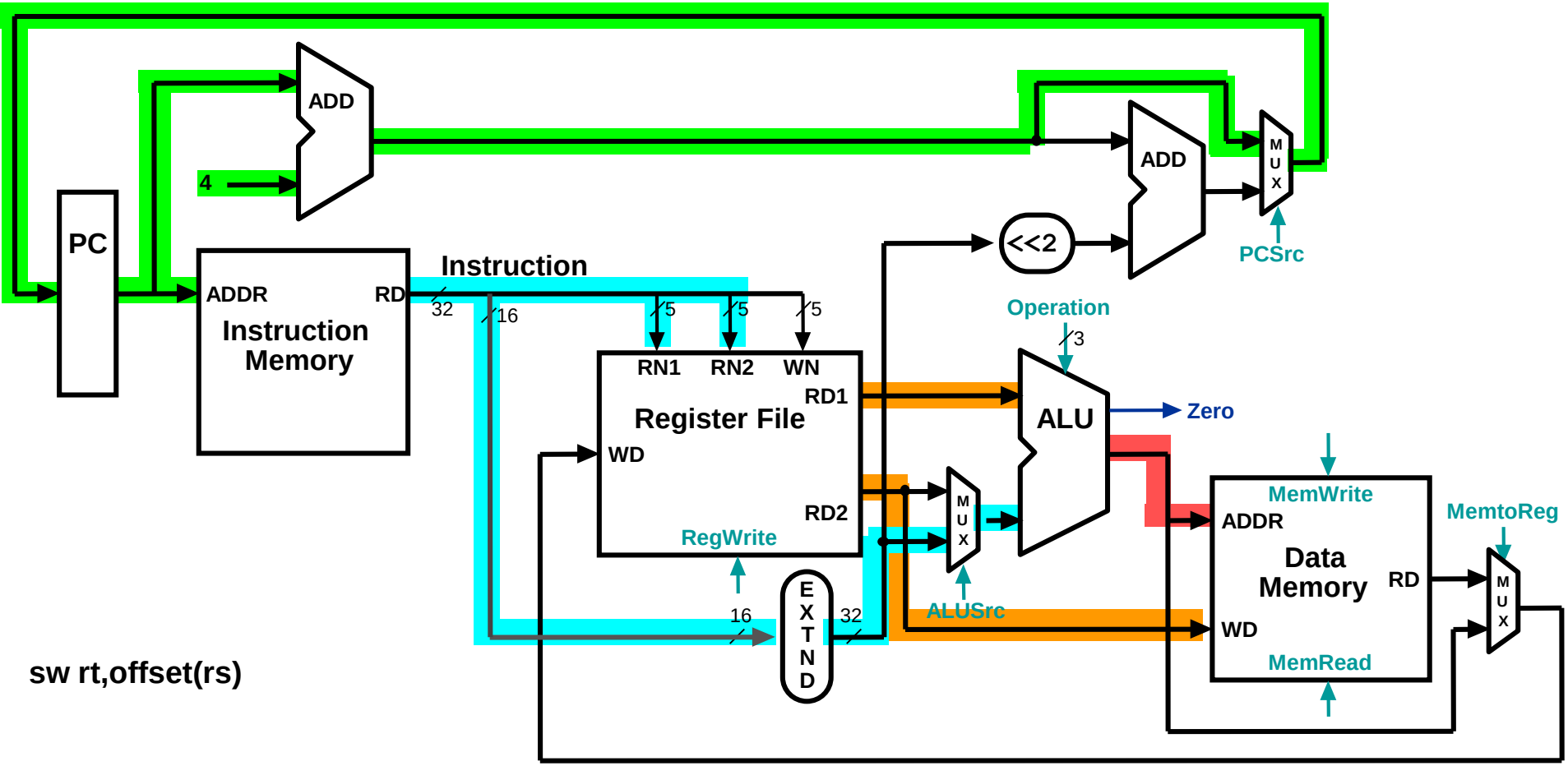
add rd, rs, rt

Datapath Executing `lw`



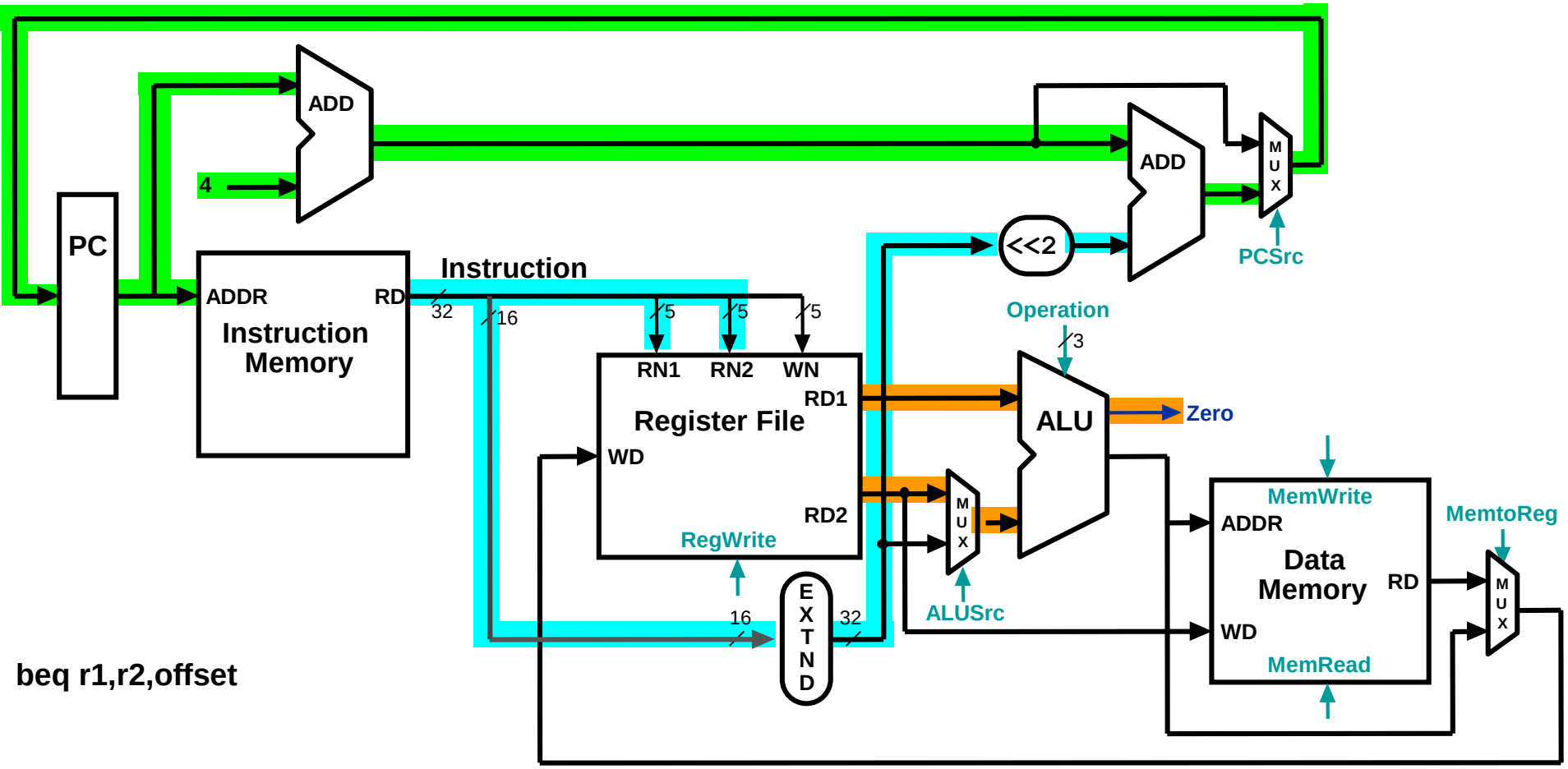
lw rt,offset(rs)

Datapath Executing `sw`



sw rt,offset(rs)

Datapath Executing `beq`



beq r1,r2,offset

Q A new *add* instruction is to be included in the MIPS ISA. The R-type format is used for coding the instruction and the instruction does the following:

ADD Rd, Rs, [Rt]: adds the contents of Register Rs with the content of memory pointed by Rt and store the result in Rd.

Show what changes are needed to support this *ADD* instruction.
Do not lengthen the clock cycle. Do not add any additional functional unit.